PROJECT 24

AI-Driven Smart Heritage Site Preservation & Monitoring System

Intelligent Cultural Heritage Intelligence Platform Using AI and IoT for Conservation and Risk Detection

Anishka Jain
IoT & AI Lead

Bhoomika Agarwal
Cloud & Backend Lead

Shraddha Singh
Dashboard & Docs Lead

Industry: Archaeological Orgs, Govt Heritage Bodies, Cultural Preservation Firms

Category: AI + IoT + Cloud Platform

Team Size: 3 Members (Equal Contribution)

Duration: 4 Months | 16 Weeks

33.3%
Anishka Jain
IoT System + AI Engine

33.3%
Bhoomika Agarwal
Cloud Architecture + Backend API

33.4%
Shraddha Singh
Dashboard + Docs + QA

PROJECT OVERVIEW & COMPONENT MAPPING

System architecture breakdown and responsibility assignment for all three modules

This document defines the complete workload division for Project 24. Each member owns one primary module. All three modules are tightly integrated and interdependent — the system cannot function without all three.

System Component	Description	Primary Lead
IoT Sensing Layer	Environmental & structural sensor network, edge data acquisition and preprocessing	**Anishka Jain**
AI Risk Engine	ML models for deterioration prediction & vandalism anomaly detection	**Anishka Jain**
Cloud Infrastructure	AWS/Azure deployment, databases, IoT data ingestion pipeline, security	**Bhoomika Agarwal**
Alert & Reporting System	Real-time multi-channel alerts (email/SMS/push) and automated PDF reports	**Bhoomika Agarwal**
Conservation Dashboard	Interactive React.js UI, data visualization, GIS mapping, Site Health Index	**Shraddha Singh**
Documentation & QA	System docs, test plans, user manual, cultural preservation strategy document	**Shraddha Singh**

MEMBER 1 — ANISHKA JAIN

IoT System Architecture & AI/ML Risk Assessment Engine | Contribution: 33.3%

MEMBER 01

Anishka Jain

IoT System Architect & AI/ML Engineer

Project Contribution: 33.3% | Module: IoT Sensing Layer + AI Risk Engine

DOMAIN FOCUS

IoT Hardware Integration, AI/ML Model Development, Sensor Data Engineering, Edge Computing, Time-Series Analysis

Python | TensorFlow/Keras | MQTT/CoAP | Raspberry Pi | Scikit-learn | XGBoost | Edge AI | LoRaWAN

CORE RESPONSIBILITIES

IoT Architecture Design	Design complete sensor network topology for heritage site monitoring. Select sensors: DHT22 (temperature/humidity), MQ-series (air quality), accelerometers (vibration), UV sensors, crack displacement sensors. Define MQTT/CoAP protocols and edge preprocessing pipelines for remote sites.
Sensor Hardware Setup	Configure Raspberry Pi / Arduino IoT nodes. Program sensor firmware in MicroPython/C++. Implement edge-level noise filtering, data normalization, and local buffering. Set up LoRaWAN/Wi-Fi wireless communication for heritage sites with limited connectivity.
Data Preprocessing Pipeline	Build data ingestion pipelines from IoT nodes to cloud. Handle missing sensor values, outliers, and time-series alignment. Create feature engineering scripts extracting deterioration-relevant features: temperature cycling stress, humidity gradients, vibration frequency signatures.
AI Deterioration Model	Develop and train supervised ML models — Random Forest, LSTM (time-series), XGBoost — to predict stone/wood/metal deterioration rates from environmental exposure history. Use historical conservation records + synthetic simulation data. Target: AUC greater than 0.88 on validation set.
Vandalism Risk Detection	Build anomaly detection module using Isolation Forest to flag unusual activity patterns: unauthorized access, sudden structural shocks, night-time vibration anomalies. Integrate detection output with Bhoomika's alert pipeline via REST API calls on risk threshold breach.
Model Evaluation	Conduct cross-validation, confusion matrix analysis, and feature importance ranking. Generate Risk Prediction Performance Report with AUC, precision, recall, F1-score. Document hyperparameter tuning process. This is a required project expected outcome.
Cloud Integration	Collaborate with Bhoomika to ensure IoT streams are correctly ingested via AWS IoT Core. Test end-to-end flow: sensor node to cloud to AI prediction output. Expose trained model as REST API endpoint consumed by both backend (for alerts) and dashboard (for visualization).

Key Deliverables

- ✓ IoT Sensor Network Architecture Diagram
- ✓ Configured sensor nodes with firmware code
- ✓ Data preprocessing & feature engineering scripts
- ✓ Trained AI deterioration prediction model
- ✓ Vandalism anomaly detection module

Technologies & Tools

- ▸ Python 3.x — primary language
- ▸ TensorFlow / Keras — LSTM deep learning
- ▸ Scikit-learn — Random Forest, Isolation Forest
- ▸ XGBoost — gradient boosting
- ▸ MQTT / CoAP — IoT communication

- ✓ Risk Prediction Performance Report
- ✓ Model training notebooks & evaluation metrics
- ✓ REST API endpoint for AI predictions

- ▸ Raspberry Pi 4 / Arduino — edge hardware
- ▸ LoRaWAN — long-range wireless
- ▸ AWS IoT Core — cloud IoT gateway

Phase 1: Sensor Setup	Phase 2: Data Pipeline	Phase 3: Model Training	Phase 4: Integration	Phase 5: Evaluation
Weeks 1–3	Weeks 4–6	Weeks 7–11	Weeks 12–14	Weeks 15–16

MEMBER 2 — BHOOMIKA AGARWAL

Cloud Infrastructure, Backend REST API & Real-Time Alert System | Contribution: 33.3%

MEMBER 02

Bhoomika Agarwal

Cloud Architect & Backend API Developer

Project Contribution: 33.3% | Module: Cloud + Backend API + Alert System

DOMAIN FOCUS

Cloud Infrastructure Design, RESTful API Development, Database Engineering, Real-time Event Systems, Cybersecurity, CI/CD Automation

AWS/Azure | FastAPI/Node.js | PostgreSQL | Docker/K8s | Apache Kafka | CI/CD | JWT/RBAC | Redis

CORE RESPONSIBILITIES

Cloud Architecture Design	Design and deploy secure, scalable cloud architecture on AWS. Set up VPCs, IAM roles, auto-scaling groups, and load balancers. Services include: AWS IoT Core (MQTT ingestion), RDS PostgreSQL (structured data), TimescaleDB (time-series sensor data), S3 (raw sensor dumps), Lambda (serverless event triggers).
Database Schema & Management	Design relational database schema for sensor readings, site metadata, AI predictions, alert logs, and user accounts. Implement TimescaleDB for efficient time-series storage of high-frequency sensor data. Set up automated backups, read replicas, data retention policies, and partitioning for performance.
RESTful API Development	Build comprehensive FastAPI / Node.js REST API with endpoints for: sensor data ingestion, AI model result retrieval, alert management, user authentication, site management, and automated report generation. Full OpenAPI/Swagger documentation. Versioned endpoints for future extensibility.
Real-time Alert System	Build event-driven alert pipeline using Apache Kafka / AWS SNS+SQS. When Anishka's AI model flags a risk, trigger multi-channel notifications: email (AWS SES), SMS (Twilio), push notifications, and in-app alerts. Implement four severity levels: Low, Medium, High, Critical with escalation logic.
Security & Authentication	Implement JWT-based authentication and Role-Based Access Control (RBAC) for conservation authorities, government users, and field operators. Set up HTTPS/TLS encryption, API rate limiting, SQL injection prevention, input validation, and audit logging. Ensure full OWASP Top 10 compliance.
Data Pipeline Orchestration	Set up Apache Airflow / AWS Step Functions to orchestrate daily batch jobs: model retraining triggers, automated report generation, data archival tasks, and database maintenance. Monitor pipeline health with CloudWatch. Alert on pipeline failures within 5 minutes.
Integration & Load Testing	Integrate Anishka's AI prediction API results into the backend pipeline. Provide stable, documented API endpoints to Shraddha for dashboard consumption. Write integration and load tests using Locust — ensuring system handles 500+ concurrent IoT device connections without performance degradation.

Key Deliverables

- ✓ Cloud architecture diagram (AWS/Azure)
- ✓ Database schema — ER diagram + migrations
- ✓ REST API with full Swagger documentation

Technologies & Tools

- ▸ AWS — IoT Core, RDS, S3, Lambda, SNS
- ▸ FastAPI (Python) / Node.js — backend
- ▸ PostgreSQL + TimescaleDB — database

- ✓ Real-time multi-channel alert system
- ✓ JWT authentication + RBAC system
- ✓ CI/CD pipeline (GitHub Actions)
- ✓ Load testing report (500+ connections)
- ✓ Security audit & compliance report

- ▸ Apache Kafka — event streaming
- ▸ Docker + Kubernetes (EKS)
- ▸ GitHub Actions — CI/CD automation
- ▸ Redis — caching layer
- ▸ Locust — load & stress testing

Phase 1: Architecture	Phase 2: DB & API	Phase 3: Alert System	Phase 4: Security/CI	Phase 5: Testing
Weeks 1–3	Weeks 4–7	Weeks 8–10	Weeks 11–13	Weeks 14–16

MEMBER 3 — SHRADDHA SINGH

Conservation Dashboard, Technical Documentation & QA Testing | Contribution: 33.4%

MEMBER 03

Shraddha Singh

Frontend Developer & Documentation Lead

Project Contribution: 33.4% | Module: Dashboard + Docs + QA + Coordination

DOMAIN FOCUS

Frontend Development, UI/UX Design, Data Visualization, GIS Mapping, Technical Documentation, QA Testing, Project Coordination

React.js | D3.js/Recharts | Leaflet/Mapbox | Figma | Cypress/Jest | GIS Mapping | Tech Writing | TypeScript

CORE RESPONSIBILITIES

Dashboard UI/UX Design	Design wireframes and high-fidelity Figma mockups for the cloud-based conservation dashboard. Multi-panel layout: real-time sensor heatmaps, site health score gauges, risk level indicators, alert timelines, historical trend charts, GIS site maps. Ensure WCAG 2.1 accessibility and full responsiveness for tablet and desktop.
Frontend Development	Implement full dashboard frontend in React.js with TypeScript. Integrate with Bhoomika's REST API using Axios/React Query. Implement WebSocket connections for real-time sensor data updates without page refresh. Use Redux Toolkit for state management across all dashboard panels. Build reusable component library in Storybook.
Data Visualization & GIS	Build interactive charts using Recharts and D3.js: temperature trend lines, vibration spectra, humidity heatmaps, AI risk score time series, alert frequency histograms. Integrate Leaflet.js / Mapbox GL for geographic heritage site mapping with dynamic risk level colour overlays and multi-site clustering views.
Site Health Index (SHI)	Develop the Site Health Index scoring system — a composite metric combining environmental sensor data, structural readings, and AI risk scores into a single 0–100% health percentage per site. Visualize SHI trends over time. Build cross-site comparison panels enabling conservation authorities to prioritize intervention locations.
QA Testing	Write end-to-end test suites using Cypress for all dashboard user workflows: login, alert management, site health views, report downloads. Perform Jest unit testing for React components. Cross-browser testing across Chrome, Firefox, Safari, and Edge. Conduct full system integration testing coordinating with Anishka and Bhoomika.
Technical Documentation	Author complete system documentation: System Architecture Document, API Usage Guide, IoT Setup Manual, and User Manual for conservation authorities. Create UML diagrams — class, sequence, and deployment diagrams. Write the Cultural Preservation Strategy Document, a key required project deliverable for the expected outcomes list.
Project Coordination	Maintain the Gantt chart and track milestone completion across all three members. Facilitate weekly stand-up meetings. Prepare final project presentation slides and demo video. Compile the Risk Prediction Performance Report using Anishka's model results. Ensure all deliverables are complete and submission-ready by Week 16.

Key Deliverables

- ✓ Figma wireframes & UI/UX prototypes
- ✓ Functional conservation dashboard (React.js)

Technologies & Tools

- ▸ React.js + TypeScript — frontend framework
- ▸ Recharts / D3.js — data visualization

- ✓ GIS site maps with risk overlays
- ✓ Site Health Index (SHI) analytics module
- ✓ Cypress E2E test suite + Jest unit tests
- ✓ System Architecture Document (UML)
- ✓ Cultural Preservation Strategy Document
- ✓ User Manual for conservation authorities
- ✓ Final presentation deck & demo video

- ▸ Leaflet.js / Mapbox GL — GIS mapping
- ▸ Tailwind CSS — UI styling
- ▸ Redux Toolkit — state management
- ▸ WebSocket — real-time data streaming
- ▸ Cypress — E2E testing automation
- ▸ Figma — UI/UX design tool

Phase 1: UI/UX Design	Phase 2: Core Dashboard	Phase 3: Charts & GIS	Phase 4: Testing & Docs	Phase 5: Final Polish
Weeks 1–3	Weeks 4–7	Weeks 8–10	Weeks 11–14	Weeks 15–16

TEAM COLLABORATION MATRIX & MILESTONE CALENDAR

Integration touchpoints and shared delivery schedule across all three modules

Integration Points

Integration Point	Anishka Jain	Bhoomika Agarwal	Shraddha Singh
IoT Data to Cloud	Provides MQTT data stream	Configures IoT Core intake	Monitors live in dashboard
AI Predictions to API	Exposes model as REST endpoint	Routes predictions to DB and alerts	Displays risk scores in UI
Alert Trigger	Sets risk thresholds in model	Sends via Kafka/SNS/Twilio	Shows alert timeline in UI
System Testing	Tests sensor + model accuracy	Tests API load and security	Tests full user workflows (E2E)
Documentation	IoT setup + AI technical specs	API guide + cloud architecture	User manual + strategy doc
Final Demo	Live sensor feed + model output	Cloud console + API demo	Full dashboard walkthrough

16-Week Project Milestone Calendar

Milestone	Target Week	Owner
IoT nodes operational and sending live sensor data	Week 4	Anishka
Cloud infrastructure fully deployed on AWS	Week 4	Bhoomika
End-to-end data pipeline: sensor to database working	Week 7	Anishka + Bhoomika
AI deterioration model trained (target AUC > 0.85)	Week 11	Anishka
REST API complete with full Swagger documentation	Week 10	Bhoomika
Multi-channel alert system live and tested	Week 11	Bhoomika
Conservation Dashboard v1 launched with core panels	Week 11	Shraddha
GIS site maps + Site Health Index analytics live	Week 13	Shraddha
Full system integration testing complete	Week 14	All Three Members
All documentation complete and reviewed	Week 15	Shraddha (lead)
Risk Prediction Performance Report submitted	Week 15	Anishka + Shraddha
Final demo recording and project submission	Week 16	All Three Members

Expected Outcomes vs Deliverables Mapping

Required Expected Outcome	Deliverable	Lead	Support
Functional AI-powered heritage preservation system	Integrated system: IoT + AI + Cloud + Dashboard	All three	—
Conservation analytics dashboard	React.js dashboard: charts, GIS, SHI module	Shraddha	Bhoomika (API)
Risk prediction performance report	Model eval: AUC, precision, recall, F1, confusion matrix	Anishka	Shraddha (write-up)

Cultural preservation strategy document	Strategy doc with recommendations and site priorities	**Shraddha**	Anishka (data)
Alert and reporting system	Multi-channel alert pipeline + automated PDF reports	**Bhoomika**	Shraddha (UI alerts)
Secure and scalable architecture	AWS deploy: IAM, HTTPS, auto-scaling, audit logs	**Bhoomika**	—

All three members contribute equally and complementarily — no single module works without the others.
Anishka's IoT sensors and AI models generate the intelligence. Bhoomika's cloud backend stores, processes, and distributes it securely with real-time alerts. Shraddha's dashboard makes it visible and actionable for conservation authorities. Together they form a complete, end-to-end AI-driven heritage preservation platform.